MongoDB 8.0 · SEGURIDAD TEORIA

Edición contrastada con los scripts de GitHub Actions. Las cifras de explain() son medidas de una ejecución concreta, no compromisos de rendimiento. Las actividades no cubiertas por el CI se identifican como ampliación hasta tener pruebas individuales.

Seguridad

Teoría del capítulo 6

Una arquitectura de controles: credenciales, permisos, TLS, certificados, LDAP y cifrado de datos.

Base técnica de referencia: repositorio MongoDB y evidencias de ejecución. Los códigos se estudian en `mongosh`, con una base preparada y, si necesitan usuarios/índices previos, después del paso correspondiente. Las variantes de diseño que no están incluidas en GitHub Actions se consideran ejercicios de análisis o ampliación, no resultados ejecutados.

Mapa de seguridad

Identidad, privilegios, canal y datos persistidos

Cuatro preguntas que organizan el capítulo

- ¿Puede el cliente llegar al servidor? Aislamiento de red y publicación de puertos.
- ¿Quién es? SCRAM o certificados cuando estén configurados.
- ¿Qué puede hacer? Roles de base de datos y prueba de operaciones denegadas.
- ¿Qué información queda protegida? TLS, cifrado en reposo, secretos y copias.

Comprobaciones de la sesión

Se utilizan dos instancias locales: la de índices sin autenticación limitada a localhost y la de seguridad con usuario administrador y lector. Se comparan explícitamente una lectura sin credenciales que se rechaza, una lectura permitida y una escritura rechazada. Son pruebas del lado servidor, no simples capturas de Compass.

RED → IDENTIDAD → ROL → DATOS

Qué queda fuera del laboratorio esencial

LDAP en despliegue autogestionado depende de Enterprise y está obsoleto desde MongoDB 8.0. El cifrado en reposo del motor WiredTiger autogestionado también es Enterprise. TLS y X.509 se desarrollan conceptualmente y existe una ampliación optativa de certificados con requisitos propios.

Referencias oficiales

https://www.mongodb.com/docs/manual/core/security-x.509/

https://www.mongodb.com/docs/v8.0/core/security-ldap-external/

https://www.mongodb.com/docs/manual/core/security-encryption-at-rest/

https://www.mongodb.com/docs/v8.0/tutorial/configure-ssl/

> Una cuenta de clase y una CA local son ejemplos didácticos. No sirven como configuración de producción ni reemplazan una política de seguridad.

01. Seguridad por capas

Explicación, evidencia y criterio técnico

Concepto y mecanismo

La seguridad combina aislamiento de red, identidad, autorización, cifrado, gestión de secretos, copias y monitoreo; ninguna capa sustituye a otra.

Imagina una base de ventas con tres puertas: la red permite llegar al servidor, la autenticación identifica al interlocutor y la autorización decide qué puede hacer. TLS protege lo que viaja entre cliente y servidor; el cifrado en reposo protege copias de archivos según su amenaza. Ninguna puerta sustituye a las otras. Por eso la misma práctica hará fallar una lectura anónima y una escritura con credenciales válidas pero insuficientes. Las dos denegaciones tienen causas y remedios diferentes.

Caso que resuelve

RED → IDENTIDAD → PERMISO

Servidor escuchando solo en localhost con autenticación habilitada. Diseñar controles que funcionen aunque otro falle.

Demostración o comprobación

```
docker compose ps
```

Segunda situación para contrastar

```
docker compose ps
```

El servicio con permiso root no está protegido solo por estar en localhost. La red reduce exposición, SCRAM identifica y los roles restringen; son controles independientes.

Cómo interpretar el resultado

Un puerto limitado a localhost reduce exposición, pero no sustituye autenticación; un proceso local con acceso al puerto podría intentar operaciones.

> **Precaución técnica.** Confiar solo en una contraseña es insuficiente.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Servidor escuchando solo en localhost con autenticación habilitada.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

02. Modelo de amenazas

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Un modelo de amenazas comienza por datos, actores, superficies y consecuencias. Diferenciar laboratorio de producción evita falsos supuestos.

Antes de tocar una opción, pregunta qué activo se protege y de quién. Un usuario accidental, un proceso local malicioso, un intruso en la red y el robo de un disco son escenarios distintos. La tabla amenaza → superficie → control → prueba demuestra que una casilla de configuración no equivale a seguridad global. En clase, los datos son sintéticos y los contenedores están en localhost; la amenaza que realmente se reproduce es el intento de acceso sin permiso, no una intrusión remota real.

Caso que resuelve

Atacante intenta leer ventas de clientes desde una red externa. Enumerar activos, amenazas, medidas y pruebas verificables.

Demostración o comprobación

```
docker compose config
```

Segunda situación para contrastar

```
docker compose config
```

El resultado permite revisar puertos y volúmenes. Es inspección de configuración, no una prueba de intrusión ni una auditoría de producción.

Cómo interpretar el resultado

Evaluar usuario malicioso, conexión interceptada y robo del volumen requiere controles diferentes; una sola prueba no valida todos.

> **Precaución técnica.** Una conexión bloqueada por red no prueba que la base autentique correctamente.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Atacante intenta leer ventas de clientes desde una red externa.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

03. Autenticación vs autorización

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Autenticación pregunta quién se conecta; autorización determina qué operaciones puede realizar. Son pasos distintos.

La autenticación responde «¿quién eres?» y la autorización «¿puedes ejecutar esta operación?». Es posible que el cliente abra una conexión TCP y vea el shell, pero falle después al ejecutar find. También es posible que usuario y contraseña sean correctos y que insertOne sea rechazado con Unauthorized. Son dos evidencias diferentes: éxito del inicio de sesión y comprobación de los privilegios. Separarlas evita diagnosticar un error de rol como si fuera una contraseña incorrecta.

Caso que resuelve

LOGIN → ROL → OPERACIÓN

Un alumno válido puede consultar documentos pero no eliminarlos. Hacer una prueba de identidad y otra de permisos.

Demostración o comprobación

```
db.runCommand({connectionStatus:1})
```

Segunda situación para contrastar

```
db.getSiblingDB("curso_seguridad").secretos.findOne()
```

La misma lectura será autorizada para lectorlab y denegada para la sesión anónima. La conexión TCP puede haberse establecido en ambos casos.

Cómo interpretar el resultado

La operación sin login debe fallar al consultar la colección protegida; una conexión establecida solo prueba la capa de transporte.

> **Precaución técnica.** Una conexión TCP exitosa no significa que se haya autenticado.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Un alumno válido puede consultar documentos pero no eliminarlos.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

04. MongoDB Community y Enterprise

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Community ofrece autenticación SCRAM, TLS, roles y otras capacidades básicas; Enterprise añade integraciones empresariales y cifrado nativo en reposo.

La edición Community permite construir una demostración genuina de SCRAM, roles y conectividad. TLS también puede configurarse. Las características empresariales como integración LDAP de MongoDB Enterprise y cifrado nativo WiredTiger en reposo no aparecen por publicar un puerto Docker. X.509 requiere un laboratorio PKI con certificados válidos; no debe presentarse una cadena teórica como si hubiera quedado ejecutada. Para cada función la guía diferencia disponible, requisitos adicionales y evidencia verificable.

Caso que resuelve

Lab local con dos instancias Community, una abierta solo en localhost y otra con --auth. Elegir una práctica viable sin licencias ni servicios corporativos.

Demostración o comprobación

```
db.version()
```

Segunda situación para contrastar

```
db.version()
```

Verificar la versión del servidor es útil para saber qué funciones existen. La disponibilidad de una edición no se deduce solo del número de versión.

Cómo interpretar el resultado

El laboratorio Community comprueba SCRAM y roles; no demuestra LDAP Enterprise ni WiredTiger cifrado en reposo.

> **Precaución técnica.** No atribuir funciones Enterprise a una imagen Community.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Lab local con dos instancias Community, una abierta solo en localhost y otra con --auth.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

05. Usuarios, bases y authSource

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Un usuario existe en una base de autenticación concreta. La base contra la que se autentica puede diferir de la base consultada.

En MongoDB, el nombre de usuario está ligado a la base donde se creó. adminlab se autentica en admin; lectorlab, creado en curso_seguridad, lo hace con authSource=curso_seguridad. Probar una clave correcta con authSource equivocado puede fallar; eso no significa que la cuenta no exista. Los roles concedidos pueden referirse a bases diferentes de la que almacena la identidad. En una URI, authSource identifica el origen de credenciales y no es un cambio de autorización.

Caso que resuelve

USUARIO → AUTHSOURCE → ROL

Usuario lectorlab creado en curso_seguridad y no en admin. Configurar authSource exactamente donde se creó el usuario.

Demostración o comprobación

```
db.getSiblingDB("curso_seguridad").getUsers()
```

Segunda situación para contrastar

```
db.runCommand({connectionStatus:1})
```

La información de autenticación efectiva puede ser distinta a la base de datos seleccionada con use. Es necesario conocer la base donde se registró el usuario.

Cómo interpretar el resultado

authSource=curso_seguridad es correcto para lectorlab; con admin, aun con clave válida, no coincide la identidad esperada.

> **Precaución técnica.** Usar authSource=admin por costumbre provoca fallos cuando el usuario está en otra base.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Usuario lectorlab creado en curso_seguridad y no en admin.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

06. SCRAM y credenciales

Explicación, evidencia y criterio técnico

Concepto y mecanismo

SCRAM evita enviar la contraseña en claro a nivel de mecanismo de autenticación, pero no sustituye TLS para proteger el canal.

SCRAM es un mecanismo de autenticación basado en secretos compartidos y desafío-respuesta. La contraseña no se transmite como texto literal en cada validación del protocolo, pero la existencia de SCRAM no cifra automáticamente el resto del tráfico de aplicación. Una consulta que viaja sin TLS puede exponer datos sensibles a un observador de red. Por eso el diseño correcto combina autenticación con protección del transporte, almacenando secretos de modo seguro y rotando credenciales cuando corresponde.

Caso que resuelve

Autenticación de un cliente con usuario y contraseña. Proteger canal y secretos además del mecanismo de autenticación.

Demostración o comprobación

```
docker compose exec -it mongo-seguridad mongosh --host localhost --port 27017 -u lectorlab --authenticationDatabase curso_seguridad
```

Segunda situación para contrastar

```
db.runCommand({connectionStatus:1})
```

Una sesión autenticada puede seguir transmitiendo información sin TLS. El mecanismo SCRAM y la protección de transporte contestan preguntas diferentes.

Cómo interpretar el resultado

SCRAM intercambia pruebas criptográficas, pero sin TLS el tráfico de aplicación y metadatos de transporte no quedan protegidos por cifrado del canal.

> **Precaución técnica.** No introducir contraseñas reales en archivos entregados a alumnos.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Autenticación de un cliente con usuario y contraseña.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

07. Activar la autenticación

Explicación, evidencia y criterio técnico

Concepto y mecanismo

La autorización debe habilitarse en el servidor y probarse mediante operaciones denegadas. El proceso de primer usuario requiere planificar el arranque.

Activar authorization hace que MongoDB compruebe permisos. El contenedor oficial puede crear un administrador inicial al arrancar por primera vez con las variables MONGO_INITDB_ROOT_*, siempre que el directorio de datos esté vacío. Cambiar el archivo .env después no reinicializa cuentas dentro de un volumen existente. Es una fuente habitual de errores en clase. No se debe sugerir borrar volúmenes de forma general: solo los volúmenes nombrados del laboratorio y con aviso de pérdida de datos.

Caso que resuelve

Contenedor oficial inicializado con MONGO_INITDB_ROOT_USERNAME/PASSWORD. Verificar que el volumen sea nuevo al inicializar usuarios por primera vez.

Demostración o comprobación

```
docker compose exec mongo-seguridad mongosh --eval "db.version()"
```

Segunda situación para contrastar

```
docker compose logs mongo-seguridad --tail 30
```

Los mensajes de inicialización permiten comprobar que el proceso arrancó, pero no sustituyen una lectura real bajo la cuenta creada.

Cómo interpretar el resultado

Las variables MONGO_INITDB_ROOT_* solo crean la identidad inicial en un directorio de datos nuevo; cambiar variables no rota cuentas existentes.

> **Precaución técnica.** Cambiar .env sobre un volumen ya creado no regenera la contraseña root.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Contenedor oficial inicializado con MONGO_INITDB_ROOT_USERNAME/PASSWORD.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

08. Roles predefinidos

Explicación, evidencia y criterio técnico

Concepto y mecanismo

read, readWrite, dbAdmin, userAdmin y roles de alcance amplio obedecen necesidades distintas.

read da acceso a operaciones de consulta en la base indicada; readWrite añade operaciones de modificación de datos. Los roles administrativos permiten acciones mucho más amplias. Crear un usuario único para todas las tareas del curso sería cómodo pero impediría comprobar la separación de privilegios. Documenta roles y bases de datos, no una clasificación moral del usuario: el mismo usuario puede tener distintos privilegios en distintas bases.

Caso que resuelve

Alumnos con read; administrador solo para preparar prácticas. Asignar el menor privilegio requerido por caso de uso.

Demostración o comprobación

```
db.getSiblingDB("curso_seguridad").getRoles({showBuiltinRoles:true})
```

Segunda situación para contrastar

```
db.getSiblingDB("curso_seguridad").getUser("lectorlab")
```

El administrador debe observar que el rol está limitado a read sobre la base de formación. Una cuenta admin no es necesaria para la aplicación lectora.

Cómo interpretar el resultado

read permite find y no insert, update ni delete; readWrite expande permisos dentro de su base de datos designada.

> **Precaución técnica.** No usar root en aplicaciones, salvo tareas administrativas controladas.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Alumnos con read; administrador solo para preparar prácticas.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

09. Mínimo privilegio

Explicación, evidencia y criterio técnico

Concepto y mecanismo

El principio de menor privilegio reduce el impacto de fuga de credenciales, errores y consultas indebidas.

El mínimo privilegio se prueba con una cuenta que puede leer secretos_demo y no puede insertar en ella. Si ambos comandos funcionan, el laboratorio ha concedido demasiado. Si ambos fallan, no se ha demostrado una política de solo lectura, sino un problema de autenticación o contexto. La prueba positiva es tan importante como la negativa: tiene que existir una operación autorizada que confirme que la sesión realmente se autenticó.

Caso que resuelve

READ → INSERT → DENEGADO

Usuario que lee informes pero no modifica pagos. Probar una operación permitida y otra denegada.

Demostración o comprobación

```
db.secretos.findOne()
```

Segunda situación para contrastar

```
db.getSiblingDB("curso_seguridad").secretos.insertOne({prueba:"no"})
```

Esta inserción, realizada por un usuario read, debe fallar. Verifica con una lectura que no hubo alteración de datos.

Cómo interpretar el resultado

La denegación de insert a lectorlab verifica autorización del lado servidor, no una restricción de interfaz o formulario.

> **Precaución técnica.** No confundir rol readWrite con permisos sobre otras bases.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Usuario que lee informes pero no modifica pagos.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

10. Separación de responsabilidades

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Conviene distinguir cuentas de servicio, cuentas humanas y administración; compartir credenciales elimina trazabilidad.

Separar la cuenta de administración de la cuenta de aplicación permite atribuir acciones, rotar secretos y limitar daños. El operador que crea roles no debería prestar sus credenciales a un programa de consulta. En producción se aplican políticas de identidades, secretos y auditoría; en clase basta con crear usuarios diferenciados y registrar para cada uno qué acciones están habilitadas. Aumentar privilegios solo para salvar un error de laboratorio elimina precisamente la lección que se intenta impartir.

Caso que resuelve

Tres usuarios distintos para lectura, escritura y operaciones de mantenimiento. Crear cuentas específicas para cada flujo y rotarlas independientemente.

Demostración o comprobación

```
db.getSiblingDB("curso_seguridad").getUsers()
```

Segunda situación para contrastar

```
db.getSiblingDB("curso_seguridad").getUsers()
```

Un inventario de identidades permite separar cuentas humanas, de aplicación y de auditoría, cada una con permisos diferentes.

Cómo interpretar el resultado

Dos cuentas con privilegios similares siguen permitiendo rotación y atribución separadas si no comparten identidades.

> **Precaución técnica.** Un login de administrador incorporado al código no es una buena práctica.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Tres usuarios distintos para lectura, escritura y operaciones de mantenimiento.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

11. Errores de acceso

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Los errores Unauthorized, Authentication failed y conexión rechazada apuntan a problemas diferentes. Diagnosticar primero la capa.

Authentication failed significa que la credencial presentada no valida la identidad esperada; Unauthorized aparece cuando una identidad ya aceptada intenta una operación no permitida. Además puede haber fallos de conexión, resolución del nombre, TLS o base de autenticación. La secuencia de diagnóstico no empieza por cambiar contraseñas: comprueba primero puerto y servidor, luego mecanismo y authSource, y por último roles. Captura el error esperado sin incrustar secretos completos en el informe.

Caso que resuelve

Sin credenciales se puede abrir shell pero se deniega lectura. Distinguir fallo de transporte, identidad y privilegio.

Demostración o comprobación

```
db.getSiblingDB("curso_seguridad").secretos.findOne()
```

Segunda situación para contrastar

```
db.getSiblingDB("curso_seguridad").secretos.findOne()
```

La prueba positiva debe preceder al error de escritura. Si falla find, no se ha demostrado una política read; existe un problema previo.

Cómo interpretar el resultado

Authentication failed señala verificación de credencial; Unauthorized señala que la operación carece de privilegio después de la conexión.

> **Precaución técnica.** No intentar arreglar un problema TLS aumentando permisos.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Sin credenciales se puede abrir shell pero se deniega lectura.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

12. Red local y bindIp

Explicación, evidencia y criterio técnico

Concepto y mecanismo

La exposición de puertos define qué equipos pueden alcanzar MongoDB. Un laboratorio abierto debe quedar limitado a 127.0.0.1.

bindIp controla las interfaces en que escucha mongod. En Docker, publicar 127.0.0.1:27018:27017 expone el servicio únicamente en el loopback del host, pero los contenedores en la red de Compose siguen teniendo otra conectividad interna. La restricción reduce superficie de red; no reemplaza la autenticación. Evita publicar la instancia libre en todas las interfaces de un equipo de aula o de una red Wi-Fi compartida.

Caso que resuelve

Puerto 27017 solo publicado en localhost del anfitrión. Comprobar binding en docker compose ps.

Demostración o comprobación

```
docker compose ps
```

Segunda situación para contrastar

```
docker compose ps
```

Comprueba que la publicación es 127.0.0.1:27018 y no 0.0.0.0:27018. Esta revisión no impide que otro proceso local intente conectarse.

Cómo interpretar el resultado

Los puertos publicados empiezan por 127.0.0.1 en Compose; cambiarlos por 0.0.0.0 ampliaría superficie de exposición.

> **Precaución técnica.** Nunca exponer una instancia sin autenticación a internet.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Puerto 27017 solo publicado en localhost del anfitrión.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

13. Definir una conexión segura

Explicación, evidencia y criterio técnico

Concepto y mecanismo

La cadena mongodb:// expresa host, puerto, credenciales y authSource; parámetros TLS completan la protección cuando está disponible.

Una conexión segura necesita llegar al servidor correcto, identificarlo, usar un canal cifrado y limitar las operaciones del usuario. Una URI con usuario y contraseña pero sin TLS puede autenticar sin proteger el tránsito. Un canal TLS con usuario root protege el transporte pero deja demasiado poder a la aplicación. En la guía se descompone la solución en conexión, canal, identidad y autorización; cada pieza tiene una prueba distinta.

Caso que resuelve

RED → TLS → ROLES

URI de Compass para lectorlab en localhost:27018. Evitar publicar URIs que contengan secretos.

Demostración o comprobación

```
mongosh "mongodb://127.0.0.1:27018/curso_seguridad"
```

Segunda situación para contrastar

```
docker compose exec -it mongo-seguridad mongosh --host localhost --port 27017
```

Llegar al servidor sin credenciales no equivale a autenticar. Un shell iniciado no debe confundir al alumno cuando find requiere permisos.

Cómo interpretar el resultado

La cadena URI puede filtrar contraseñas por shell o capturas; usar interfaz de entrada protegida en lugar de incorporarlas a documentos.

> **Precaución técnica.** Escapar caracteres reservados de contraseñas en URLs.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «URI de Compass para lectorlab en localhost:27018.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

14. Seguridad de datos en tránsito

Explicación, evidencia y criterio técnico

Concepto y mecanismo

TLS cifra y autentica el canal entre cliente y servidor, evitando exposición trivial en redes no confiables.

TLS establece un canal cifrado y autentica al servidor si el cliente valida su certificado y la cadena de confianza. El certificado del servidor debe corresponder al nombre que el cliente utiliza. En un entorno de formación puede construirse una CA local, pero esa CA no es una autoridad confiable para producción. La configuración requireTLS rechaza las conexiones que no negocian TLS; la protección de tránsito no cifra por sí misma los datos persistidos en el volumen.

Caso que resuelve

CLIENTE → TLS → SERVIDOR

Cliente TLS valida el certificado presentado por mongod. Exigir TLS en producción y validar identidad del servidor.

Demostración o comprobación

```
mongosh --tls --tlsCAFile /ruta/ca.crt
```

Segunda situación para contrastar

```
openssl x509 -in certs/server.crt -noout -subject -issuer -dates
```

Un certificado válido para el canal debe tener emisor confiable, vigencia y nombre aceptado; proteger el enlace no cifra automáticamente el volumen.

Cómo interpretar el resultado

TLS protege comunicación entre puntos; para comprobar identidad del servidor el cliente debe confiar en la CA y validar el nombre.

> **Precaución técnica.** No aceptar tlsAllowInvalidCertificates como receta habitual.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Cliente TLS valida el certificado presentado por mongod.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

15. Certificados y autoridades

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Los certificados X.509 describen identidades y vinculan claves públicas a sujetos firmados por una CA. La clave privada debe permanecer protegida.

Una autoridad de certificación firma certificados; un certificado contiene nombre, emisor, vigencia, clave pública y extensiones. El archivo PEM que utiliza el servidor puede combinar certificado y clave privada, por lo que no debe adjuntarse a repositorios ni a material compartido después de generar claves reales. El alumno puede inspeccionar subject, issuer y fechas mediante openssl sin publicar la clave. El certificado cliente necesita también restricciones de uso adecuadas.

Caso que resuelve

CA de formación emite certificado de servidor para localhost. Validar confianza, fechas y SAN del certificado.

Demostración o comprobación

```
openssl x509 -in server.crt -noout -text
```

Segunda situación para contrastar

```
openssl x509 -in certs/client.crt -noout -subject -issuer -dates
```

El subject identifica al titular de un certificado; todavía faltan la confianza de la CA y el registro como usuario externo para conceder roles.

Cómo interpretar el resultado

El certificado presenta subject, issuer, validez y SAN; firmar un certificado no asigna automáticamente roles en MongoDB.

> **Precaución técnica.** Un certificado firmado por CA desconocida no basta.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «CA de formación emite certificado de servidor para localhost.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

16. TLS: nombre del servidor

Explicación, evidencia y criterio técnico

Concepto y mecanismo

El SAN de certificado debe corresponder al nombre usado por el cliente. La confianza en CA y el nombre son controles distintos.

Si te conectas a localhost, el certificado del servidor debe contener localhost en SAN como nombre DNS; para acceder por 127.0.0.1 conviene incluir ese valor como IP SAN. Un certificado cuyo CN parece correcto pero carece del SAN apropiado puede ser rechazado. No normalices --tlsAllowInvalidHostnames como solución: en el laboratorio es mejor regenerar un certificado válido que entrenar a los alumnos a desactivar verificaciones importantes.

Caso que resuelve

Conectar por localhost a certificado para otro dominio debe fallar. Usar SAN adecuados y probar rechazos esperados.

Demostración o comprobación

```
openssl x509 -in server.crt -noout -ext subjectAltName
```

Segunda situación para contrastar

```
openssl x509 -in certs/server.crt -noout -ext subjectAltName
```

Comprueba que el SAN incluye localhost y 127.0.0.1 cuando son los destinos usados en el laboratorio.

Cómo interpretar el resultado

localhost debe figurar en SAN si se conecta así; una conexión por IP requiere además el SAN IP correspondiente.

> **Precaución técnica.** Desactivar validación anula buena parte del objetivo.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Conectar por localhost a certificado para otro dominio debe fallar.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

17. Autenticación X.509

Explicación, evidencia y criterio técnico

Concepto y mecanismo

En autenticación X.509 el certificado cliente representa al usuario y la identidad se registra habitualmente en $external. Requiere TLS.

La autenticación de cliente X.509 agrega un paso a TLS: el servidor no solo demuestra su identidad, también el cliente presenta certificado. El subject compatible con RFC2253 debe estar creado como usuario en la base $external con los roles adecuados. Sin esa correspondencia, presentar un certificado firmado no concede permiso de lectura. El laboratorio ampliado debe verificar, en este orden, cadena TLS, reconocimiento del sujeto y autorización del comando find.

Caso que resuelve

CERTIFICADO → $external → READ

Cliente se identifica con certificado personal distinto al del servidor. Relacionar DN de sujeto, usuario MongoDB y privilegios.

Demostración o comprobación

```
db.getSiblingDB("$external").getUsers()
```

Segunda situación para contrastar

```
db.getSiblingDB("$external").getUsers()
```

Si se configuró X.509, el subject del cliente debe coincidir con la identidad creada en $external. TLS exitoso sin rol no basta para hacer find.

Cómo interpretar el resultado

X.509 autentica un sujeto registrado en $external y asociado a un rol; el certificado no concede derechos solo por ser válido.

> **Precaución técnica.** No confundir usar certificado cliente para TLS con obtener permisos automáticamente.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Cliente se identifica con certificado personal distinto al del servidor.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

18. x509: preparación y rotación

Explicación, evidencia y criterio técnico

Concepto y mecanismo

La infraestructura de certificados requiere renovación, revocación práctica, distribución segura y seguimiento de caducidades.

Los certificados vencen y pueden ser revocados. Rotar un certificado cliente implica emitir uno nuevo, revisar el subject y su registro en $external, distribuir la clave privada por un canal seguro y retirar el anterior. Una CA de aula demuestra sintaxis y flujo, pero no tiene los procesos corporativos de custodia ni revocación. La renovación puede interrumpir servicios si no se ensaya; esa es la razón de documentar inventario y caducidad.

Caso que resuelve

Certificado de formación de corta vida distinto de credenciales reales. Documentar CA, claves privadas y plan de reemplazo.

Demostración o comprobación

```
openssl x509 -in client.crt -noout -dates
```

Segunda situación para contrastar

```
openssl x509 -in certs/client.crt -noout -enddate
```

Este dato se incorpora a la agenda de renovación; una llave privada robada no deja de ser peligrosa por proceder de un certificado firmado.

Cómo interpretar el resultado

Una CA local de demostración no cumple la gobernanza de certificados productivos; renovación de claves y vencimientos exige proceso.

> **Precaución técnica.** No guardar claves privadas en repositorios públicos.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Certificado de formación de corta vida distinto de credenciales reales.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

19. LDAP: identidad corporativa

Explicación, evidencia y criterio técnico

Concepto y mecanismo

LDAP es un protocolo/directorio de identidades; no equivale a cifrado. En MongoDB Enterprise ha servido para integrar autenticación y autorización externas.

LDAP permite centralizar usuarios y grupos en un directorio corporativo. En MongoDB autogestionado su integración relevante pertenece a Enterprise y está obsoleta desde la versión 8.0. No tiene sentido instalar un servidor LDAP improvisado para simular que Community ofrece una función de Enterprise. El alumnado debe comprender el flujo: cliente, servidor MongoDB, consulta al directorio, mapeo a roles y verificación de una operación. Un diagrama de ese flujo es actividad válida, pero no equivale a integración ejecutada.

Caso que resuelve

MONGODB → LDAP → GRUPOS

Un grupo LDAP de analistas se mapea a un rol MongoDB. Presentar la arquitectura y las precondiciones sin simular un LDAP inexistente.

Demostración o comprobación

```
db.runCommand({connectionStatus:1})
```

Segunda situación para contrastar

```
Diagrama: Cliente → MongoDB Enterprise → Directorio LDAP → Grupos
```

La integración requiere infraestructura y edición que no están presentes en Community. La actividad comprueba comprensión de arquitectura, no un inicio de sesión LDAP real.

Cómo interpretar el resultado

LDAP requiere servidor y edición compatibles; su obsolescencia desde MongoDB 8.0 aconseja valorar alternativas antes de proyectos nuevos.

> **Precaución técnica.** LDAP está deprecado desde MongoDB 8.0: no diseñar nuevas integraciones sin evaluar alternativas.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Un grupo LDAP de analistas se mapea a un rol MongoDB.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

20. LDAP: autorización por grupos

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Los grupos de directorio pueden mapearse a roles; implica gobierno del ciclo de vida corporativo.

En autorización LDAP, las pertenencias a grupos pueden mapearse a roles de MongoDB. Un error de mapeo que otorgue acceso de escritura a un grupo grande multiplica el riesgo. La evaluación consiste en examinar un caso concreto: AnalistasVentas → read sobre ventas; OperadoresVentas → readWrite. Después se comprueba quién gestionaría altas y bajas en el directorio y cómo se auditarían cambios en el mapeo. El modelo es conceptual sin Enterprise ni directorio de prueba.

Caso que resuelve

Salida del empleado del grupo de analistas revoca acceso indirectamente. Distinguir grupos externos de roles MongoDB.

Demostración o comprobación

```
db.getSiblingDB("admin").getRoles({showBuiltinRoles:true})
```

Segunda situación para contrastar

```
Tabla: AnalistasVentas = read; OperadoresVentas = readWrite
```

Una política de grupos debe asociar roles apropiados y prever revisión del ciclo de vida. Error en una regla afecta a todos los miembros.

Cómo interpretar el resultado

Un grupo de directorio mapeado erróneamente a privilegios amplios afecta a todos sus miembros; revisar las reglas de mapeo es esencial.

> **Precaución técnica.** Un error de mapeo puede otorgar privilegios indebidos.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Salida del empleado del grupo de analistas revoca acceso indirectamente.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

21. Alternativas modernas a LDAP

Explicación, evidencia y criterio técnico

Concepto y mecanismo

OIDC centraliza autenticación en un proveedor de identidad para despliegues compatibles, pero su disponibilidad depende de edición y servicio.

Para un diseño nuevo, la obsolescencia de LDAP obliga a evaluar mecanismos alternativos compatibles con la edición y la arquitectura elegida, como OpenID Connect donde esté admitido. No debe asumirse que cualquier contenedor local soporta cualquier proveedor de identidad. Compara cuentas locales SCRAM, autenticación con certificado y federación de identidad en términos de mantenimiento, ciclo de vida y dependencia externa, sin confundir mecanismos de autenticación con roles.

Caso que resuelve

Identidad federada frente a cuentas locales SCRAM. Enseñar comparación de capacidades y condiciones de licencia.

Demostración o comprobación

```
db.version()
```

Segunda situación para contrastar

```
Comparación: SCRAM / X.509 / OIDC / LDAP legado
```

La solución depende de edición, proveedor, política de identidades y soporte. LDAP no es opción estratégica neutra tras su obsolescencia en 8.0.

Cómo interpretar el resultado

OIDC no aparece mágicamente en el contenedor Community: enseñar el principio y las ediciones compatibles evita demos falsas.

> **Precaución técnica.** No prometer que OIDC esté disponible en la imagen Community usada.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Identidad federada frente a cuentas locales SCRAM.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

22. Encriptación en reposo

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Cifrado en reposo protege archivos en discos/volúmenes. La capacidad nativa de WiredTiger cifrado es Enterprise; Atlas dispone de funciones gestionadas.

El cifrado en reposo trata la lectura no autorizada de ficheros persistidos o copias. MongoDB Enterprise autogestionado admite cifrado nativo del motor WiredTiger; MongoDB Atlas proporciona opciones de cifrado gestionado. Guardar un volumen Docker no activa esa característica en Community. Cifrado de disco del sistema operativo, cifrado de almacenamiento en nube y cifrado nativo del motor son capas diferentes, con responsabilidades y escenarios de recuperación distintos.

Caso que resuelve

FICHEROS → CIFRADO → CLAVES

Disco robado con ficheros persistidos. Complementarlo con TLS, autenticación y claves gestionadas.

Demostración o comprobación

```
docker volume ls
```

Segunda situación para contrastar

```
docker volume ls
```

La presencia de volúmenes muestra persistencia, NO confirma cifrado. El motor WiredTiger cifrado en reposo autogestionado es una función Enterprise.

Cómo interpretar el resultado

Persistir ficheros MongoDB en Docker no los cifra automáticamente; el cifrado nativo del motor WiredTiger es característica Enterprise.

> **Precaución técnica.** El cifrado del disco no impide consultas por un usuario autorizado comprometido.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Disco robado con ficheros persistidos.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

23. Claves y KMS

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Las claves de cifrado requieren gestión de acceso, almacenamiento y rotación. La separación entre datos y claves es central.

Un sistema que guarda la clave de cifrado junto al mismo volumen de datos reduce el valor de proteger físicamente los ficheros. La administración de claves se ocupa de generación, custodia, acceso, rotación y recuperación. Las soluciones KMS separan parte de esas responsabilidades y requieren políticas de acceso y disponibilidad. Pregunta al alumnado qué ocurre si se pierde la clave al restaurar un backup: un cifrado perfectamente configurado puede hacer ilegibles datos propios sin un plan de recuperación.

Caso que resuelve

Rotar clave administrada sin almacenarla en el mismo directorio que el respaldo. Presentar KMS/KMIP como arquitectura, no como script Community falso.

Demostración o comprobación

```
docker compose config
```

Segunda situación para contrastar

```
Tabla: claves / custodio / restauración / revocación
```

Una copia cifrada sin recuperación de la clave puede ser inservible. El responsable de datos y el custodio de claves requieren decisiones separadas.

Cómo interpretar el resultado

Una clave de cifrado guardada junto a datos y copias no permite separación de funciones efectiva; KMS o equivalentes responden a esa necesidad.

> **Precaución técnica.** Un secreto incluido en la imagen Docker elimina aislamiento.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Rotar clave administrada sin almacenarla en el mismo directorio que el respaldo.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

24. Cifrado de campo a nivel cliente

Explicación, evidencia y criterio técnico

Concepto y mecanismo

El cifrado de campo del lado cliente protege campos antes de llegar al servidor bajo modelos y drivers compatibles; difiere del cifrado de disco.

El cifrado del lado del cliente puede transformar campos sensibles antes de enviarlos al servidor. Eso cambia qué ve el administrador de la base y qué consultas son posibles. No debe venderse como sustituto universal de TLS, roles ni cifrado de copia: protege otra superficie. Para un dato como documentoIdentidad, decide primero si necesita almacenarse, qué aplicación lo descifra y qué consultas deberá realizar. La elección depende del modelo de amenaza, no de habilitar una casilla genérica.

Caso que resuelve

Campo de identificación cifrado por la aplicación antes de escribirlo. Examinar amenaza que resuelve y costes operativos.

Demostración o comprobación

```
db.secretos.findOne()
```

Segunda situación para contrastar

```
Diagrama: aplicación → campo cifrado → MongoDB
```

La aplicación sabe qué descifrar. El servidor puede dejar de poder ejecutar algunas búsquedas; elegir cifrado de campo requiere estudiar consultas concretas.

Cómo interpretar el resultado

Cifrar campo antes del servidor cambia visibilidad y consultas posibles, pero no sustituye roles, TLS ni clasificación de datos.

> **Precaución técnica.** No todos los operadores de consulta son equivalentes sobre datos cifrados.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Campo de identificación cifrado por la aplicación antes de escribirlo.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

25. Gestión segura de secretos

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Los secretos deben mantenerse fuera del control de versiones, con permisos mínimos y mecanismos de rotación. .env local de laboratorio no es gestor de secretos profesional.

Escribir una clave de ejemplo en .env hace posible repetir un curso, pero no constituye una solución de secretos de producción. La contraseña de una cuenta de administrador no debería terminar en una captura, un historial de terminal o un commit. Si necesitas pasar credenciales por argumentos de consola, explica el riesgo y ofrece el modo interactivo cuando corresponda. Para formación, usa secretos sintéticos y limita todos los accesos a localhost.

Caso que resuelve

Contraseña root de práctica separada de credenciales reales. Excluir .env del ZIP público y generar valores individuales.

Demostración o comprobación

```
docker compose config --services
```

Segunda situación para contrastar

```
cat .env.example
```

El archivo contiene credenciales pedagógicas, no un secreto apto para producción. Aun así, las cadenas de conexión pueden quedar en historiales.

Cómo interpretar el resultado

La contraseña en .env demuestra parametrización, no gestión empresarial de secretos; jamás reutilizar la del laboratorio.

> **Precaución técnica.** Evitar incluir contraseñas en capturas de clase.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Contraseña root de práctica separada de credenciales reales.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

26. Registros y auditoría

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Los logs ayudan a investigar fallos; auditoría formal de base de datos es capacidad empresarial en autogestionado.

Los logs de Docker y del servidor sirven para investigar arranque, fallos de conexión y eventos visibles; no prueban que exista una auditoría regulatoria completa. Los sistemas con requisitos exigentes necesitan definir qué eventos se registran, quién puede leerlos y cuánto tiempo se conservan. Evita mostrar secretos en los mensajes de diagnóstico. El ejercicio de revisión consiste en distinguir una denegación de acceso de un fallo del contenedor, no en producir miles de líneas sin interpretación.

Caso que resuelve

Distinguir intento de conexión fallido de operación legítima. Limitar datos sensibles escritos en logs.

Demostración o comprobación

```
docker compose logs --tail=30 mongo-seguridad
```

Segunda situación para contrastar

```
docker compose logs mongo-seguridad --tail 30
```

Busca un evento relacionado con el problema observado, no un mensaje aleatorio. El registro de arranque no reemplaza auditoría formal.

Cómo interpretar el resultado

docker logs aporta diagnóstico pero no todas las evidencias de auditoría exigibles; las funciones de auditoría empresarial tienen otro alcance.

> **Precaución técnica.** Logs genéricos no equivalen a auditoría de cumplimiento.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Distinguir intento de conexión fallido de operación legítima.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

27. Backups y restauración

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Sin restauración probada, el respaldo puede dar falsa confianza. Proteger copias y credenciales de acceso.

Un backup solo resuelve el problema si puede restaurarse dentro de un objetivo aceptable de tiempo y pérdida de datos. Roles, datos, claves de cifrado y archivos de configuración pueden necesitar tratamientos diferentes. Un respaldo almacenado sin controles de acceso anula gran parte de la protección del servidor. En aula, el ejercicio correcto es planificar el ciclo de restauración sobre datos sintéticos; no se borran volúmenes sin confirmar qué contienen.

Caso que resuelve

Restaurar base de prácticas tras eliminar registros accidentalmente. Ensayar recuperación periódicamente en entornos aislados.

Demostración o comprobación

```
docker compose down
```

Segunda situación para contrastar

```
docker compose down
```

Detener servicios sin -v conserva volúmenes. Eso no es una copia de seguridad y no prueba recuperación fuera del equipo.

Cómo interpretar el resultado

Un restore probado exige claves, usuarios, copias disponibles y volumen limpio; una copia ilegible no satisface el objetivo de recuperación.

> **Precaución técnica.** Un respaldo sin cifrar puede filtrar datos aunque MongoDB esté protegido.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Restaurar base de prácticas tras eliminar registros accidentalmente.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

28. Exposición y endurecimiento

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Endurecimiento incluye bindIp, puertos mínimos, versiones soportadas, TLS, roles e inventario de dependencias.

El endurecimiento reúne servicios mínimos, bindIp, control de acceso, TLS, actualizaciones, deshabilitación de funciones innecesarias, gestión de secretos y recuperación. No existe un comando único que convierta un despliegue en seguro. Antes de abrir una interfaz de red, el equipo debe justificar quién la necesita, desde dónde y con qué identidad. La matriz de controles separa la prueba del laboratorio de un procedimiento de puesta en producción.

Caso que resuelve

Servicio docente solo en localhost y sin datos personales. Aplicar una lista de comprobación antes de abrir acceso remoto.

Demostración o comprobación

```
docker compose ps
```

Segunda situación para contrastar

```
docker compose config
```

Distingue puertos, contraseñas de laboratorio, persistencia y controles de acceso. Anota qué controles adicionales exige un despliegue remoto.

Cómo interpretar el resultado

Las restricciones de red y la autenticación deben probarse por separado; un cortafuegos mal configurado puede anular el perímetro previsto.

> **Precaución técnica.** La configuración adecuada en laboratorio no equivale a segura en producción.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Servicio docente solo en localhost y sin datos personales.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

29. Actualización y vulnerabilidades

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Mantener versiones soportadas reduce riesgo y exige plan de pruebas de compatibilidad y respaldo.

Una versión soportada recibe correcciones, pero actualizar también puede cambiar planes de ejecución, opciones de seguridad y comportamiento de cliente. En una clase reproducible conviene fijar una familia de imagen y registrar la versión efectiva; en producción conviene fijar versión exacta y mantener un calendario de parches ensayados. La política debe incluir backup, entorno de ensayo, criterios de reversión y verificación de permisos después del cambio.

Caso que resuelve

Actualización planificada de imagen Docker. Fijar versiones reproducibles en formación y planificar revisiones periódicas.

Demostración o comprobación

```
docker image inspect mongo:8.0
```

Segunda situación para contrastar

```
docker image inspect mongo:8.0
```

La etiqueta es una familia actualizable. Registra versión efectiva y planifica actualizaciones sin confundir disponibilidad de parche con instalación segura.

Cómo interpretar el resultado

Etiquetas de imagen fijadas facilitan repetición, pero hay que inspeccionar versiones reales y planificar revisión y actualizaciones.

> **Precaución técnica.** No actualizar imágenes productivas sin pruebas y rollback.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Actualización planificada de imagen Docker.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

30. Pruebas negativas controladas

Explicación, evidencia y criterio técnico

Concepto y mecanismo

La seguridad se enseña también verificando que lo prohibido falla: acceso anónimo, credencial incorrecta, operación sin privilegios.

Un test negativo debe fallar por el motivo esperado. Si un usuario read intenta insertOne y el servidor devuelve Unauthorized, el control funciona. Si el comando no conecta porque el contenedor está detenido, el fallo no prueba autorización. Primero ejecuta find como el mismo usuario y verifica que funciona; luego intenta insertOne. Esta secuencia es una prueba de seguridad mucho más fuerte que observar un error sin contexto.

Caso que resuelve

Insert como read debe devolver Unauthorized. Capturar error esperado sin convertirlo en fallo del laboratorio.

Demostración o comprobación

```
db.secretos.insertOne({prueba:"denegada"})
```

Segunda situación para contrastar

```
db.getSiblingDB("curso_seguridad").secretos.insertOne({prueba:"denegada"})
```

Si esta operación falla por Unauthorized tras una lectura exitosa, se ha verificado permiso, no un fallo de red.

Cómo interpretar el resultado

Un test de escritura rechazada cuenta como éxito de control de acceso, siempre que se descarte un error de conectividad previo.

> **Precaución técnica.** Jamás probar contra servidores ajenos o sin permiso.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Insert como read debe devolver Unauthorized.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.

31. Modelo integral de protección

Explicación, evidencia y criterio técnico

Concepto y mecanismo

Una solución defendible integra identidad, permiso, canal cifrado, cifrado de archivos, observabilidad y recuperación.

Para cerrar, piensa en una aplicación de facturación: cliente por TLS, identidad de aplicación con roles mínimos, índices para consultas frecuentes, backups comprobados y cifrado de almacenamiento según requisitos. Redacta una tabla amenaza → control → comprobación → límite. No uses la expresión «base de datos segura» sin especificar contra qué amenaza y con qué evidencia. La capacidad de argumentar los límites es un resultado de aprendizaje tan importante como saber ejecutar mongosh.

Caso que resuelve

Aplicación de ventas con rol readWrite restringido a una base. Trazar amenazas y controles por capa y probar cada una.

Demostración o comprobación

```
db.runCommand({connectionStatus:1})
```

Segunda situación para contrastar

```
db.runCommand({connectionStatus:1})
```

Cierra con una matriz de amenazas y pruebas: no se da por demostrado LDAP ni cifrado nativo solo porque SCRAM funcione.

Cómo interpretar el resultado

La matriz final debe identificar para cada amenaza un control, una verificación y una limitación, evitando expresiones absolutas como «seguro».

> **Precaución técnica.** No confundir una demo de localhost con certificación de seguridad.

Comprobación de comprensión

¿Qué resultado o evidencia confirmaría el caso «Aplicación de ventas con rol readWrite restringido a una base.»? Justifica la respuesta contrastándolo con la segunda situación. Identifica qué parte depende del diseño de datos, cuál de los índices o permisos y cuál requiere medición.